

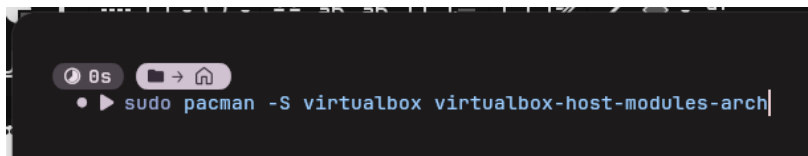
Docker 2

Preàmbul

Abans de començar la pràctica, cal mencionar que he treballat tot el curs amb Arch Linux en comptes d'Ubuntu. Se'm fa impossible instal·lar Ubuntu al pendrive perquè està molt degradat. Per aquesta raó, el procés del desenvolupament de la meva pràctica molt probablement sigui bastant diferent del d'altres alumnes, i definitivament serà diferent del que guia l'enunciat.

A més, he tingut problemes persistents amb Docker que m'impedeixen fer el pull de qualsevol imatge (hello-world, nginx, etc.), tant dins de la màquina virtual com al sistema amfitrió. La professora m'ha confirmat que puc completar l'activitat de manera teòrica, explicant les ordres, les sortides esperables i les explicacions conceptuals sense captures de pantalla.

0. Instal·lar VirtualBox



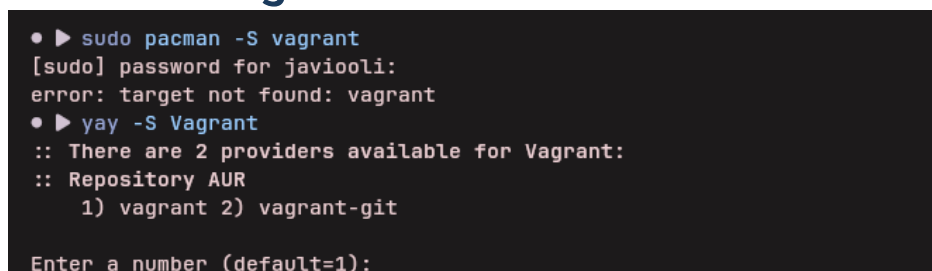
Instal·lo primer els dos paquets necessaris per poder executar VirtualBox a Arch Linux:

```
sudo pacman -S virtualbox virtualbox-host-modules-arch
```

Després reinicio i executo la ordre següent per carregar el driver de VirtualBox:

```
sudo modprobe vboxdrv
```

1. Instal·lar Vagrant



Instal·lo Vagrant amb yay, ja que al repositori oficial de pacman no està disponible. Faig servir el paquet vagrant de AUR:

```
yay -S vagrant
```

```

• ► vagrant plugin install vagrant-vbguest
Installing the 'vagrant-vbguest' plugin. This can take a few minutes.
..
Fetching micromachine-3.0.0.gem
Fetching vagrant-vbguest-0.32.0.gem
Installed the plugin 'vagrant-vbguest (0.32.0)'!
14s

```

Després instal·lo el plugin vbguest per compatibilitat amb VirtualBox (no instal·lo vagrant-libvirt, que és per KVM):

```
vagrant plugin install vagrant-vbguest
```

2. Instal·lar Docker

Instal·lo Docker al sistema amfitrió per poder comprovar i modificar coses còmodament:

```
sudo pacman -S docker docker-compose
```

```
sudo systemctl enable --now docker
```

```
sudo usermod -aG docker $USER
```

```

1s
• ► sudo pacman -S docker docker-compose

```

```

• ► sudo systemctl enable --now docker
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' → '/usr/lib/systemd/system/docker.service'.
• ► sudo usermod -aG docker $USER

```

3. Preparació final

Creo la carpeta del projecte i el fitxer variables.yml:

```
mkdir Docker2
```

```
cd Docker2
```

```

• ► cd GitHub/DAW/DAW2/DesplWeb/
• ► mkdir ./Docker2
• ► cd Docker2
0s .../DesplWeb/Docker2 ~ main

```

Contingut de variables.yml:

```
username: javiooli
```

```
password: 123456
```

```

• ► touch variables.yml
• ► nano variables.yml
• ► cat variables.yml
username: javiooli
password: 123456

```

Finalment creo el vagrantfile, corregit respecte de la pràctica basant-me en la documentació oficial de Vagrant, la de Docker a Debian i amb una mica d'ajut de la IA, perquè no aconseguia que funcionés.

Ara ja ho tenim tot llest per aixecar la màquina virtual, la aixequem amb «vagrant up» i esperem una bona estona.

```
• touch Vagrantfile
• nano Vagrantfile
• cat Vagrantfile
require 'yaml'

creds = YAML.load_file('variables.yml')

Vagrant.configure("2") do |config|

  config.vm.box = "debian/bookworm64"

  config.vm.hostname = "M11-EAC4-debian12"

  config.vm.network "private_network", ip: "192.168.11.104"

  config.vm.provider "virtualbox" do |vb|
    vb.memory = 2048
    vb.cpus = 2
  end

  config.vm.provision "shell", inline: <—SHELL

    apt update
    apt upgrade -y

    apt install -y \
      ca-certificates \
      curl \
      gnupg

    install -m 0755 -d /etc/apt/keyrings

    curl -fsSL https://download.docker.com/linux/debian/gpg \
      -o /etc/apt/keyrings/docker.asc

    chmod a+r /etc/apt/keyrings/docker.asc

    echo \
      "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
      https://download.docker.com/linux/debian \
      bookworm stable" | \
      tee /etc/apt/sources.list.d/docker.list > /dev/null

    apt update

    apt install -y \
      docker-ce \
      docker-ce-cli \
      containerd.io \
      docker-buildx-plugin \
      docker-compose-plugin

    useradd -m -s /bin/bash -G sudo,docker #{creds['username']}

    echo "#{creds['username']}:#{creds['password']}" | chpasswd

    echo "#{creds['username']} ALL=(ALL) NOPASSWD:ALL" \
      > /etc/sudoers.d/#{creds['username']}

  SHELL
end
```

4. Iniciar pràctica

Aixeco la màquina virtual amb vagrant up, i un cop llesta, hi accedeixo:

vagrant up
vagrant ssh
su javiooli
docker --version

La màquina arrenca amb Debian 12 i Docker instal·lat i funcional gràcies al provisioning del Vagrantfile. Tanmateix, he tingut problemes amb el pull d'imatges durant tres dies, però finalment ha decidit funcionar per si mateix.

```
• vagrant ssh
Linux bookworm 6.1.0-29-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.123-1 (2025-01-02) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun May 10 17:15:57 2026 from 10.0.2.2
vagrant@bookworm:~$ su javiooli
Password:
javiooli@bookworm:/home/vagrant$ docker --version
Docker version 29.4.3, build 055a478
javiooli@bookworm:/home/vagrant$ |
```

PART A - VIRTUALITZACIÓ (5 punts)

1.1 Hello World (0,5 punts)

Explicació de la ordre

```
docker run --name hola-mon hello-world
```

Aquesta ordre fa el següent, pas a pas:

Docker comprova si la imatge hello-world existeix localment al registre local de la màquina.

Si no existeix, fa un docker pull automàtic des de Docker Hub per descarregar-la.

Un cop la imatge és disponible, crea un contenidor nou a partir d'ella.

Executa el procés definit a la imatge, que escriu un missatge informatiu a la sortida estàndard i finalitza.

Com que el procés ha acabat, el contenidor s'atura (passa a l'estat Exited).

--name

Assigna un nom personalitzat al contenidor. Sense aquest argument, Docker n'assigna un d'aleatori (per exemple: «vibrant_euler»). Amb un nom propi és molt més fàcil referir-s'hi en ordres posteriors (docker stop, docker rm, docker logs, etc.).

hola-mon

És el nom que donem al contenidor. Serà identificable a docker ps -a com a «hola-mon». No és la imatge, és el nom del contenidor concret que creem a partir de la imatge.

hello-world

És el nom de la imatge Docker que es farà servir per crear el contenidor. Docker la busca primer localment i, si no hi és, la descarrega de Docker Hub.

hello-world a Docker Hub

La imatge hello-world és la imatge oficial de Docker Inc., accessible a:

```
https://hub.docker.com/\_/hello-world
```

Es tracta d'una imatge minimalista (~13 KB) dissenyada únicament per verificar que Docker funciona correctament a la màquina. L'etiqueta per defecte és latest. El seu únic propòsit és mostrar el missatge de benvinguda i sortir.

Sortida esperada a terminal

```
Unable to find image 'hello-world:latest' locally
```

```
latest: Pulling from library/hello-world
```

```
Digest: sha256:...
```

```
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
```

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

Sortida real a terminal

```
javioli@M11-EAC4-debian12:/home/vagrant$ docker run --name hola-mon hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760fd9bc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
```

1.2 Desplegar nginx

Executar docker run nginx (Terminal A)

```
docker run nginx
```

```
2026/05/13 17:34:41 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-17)
2026/05/13 17:34:41 [notice] 1#1: OS: Linux 6.1.0-47-amd64
2026/05/13 17:34:41 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/13 17:34:41 [notice] 1#1: start worker processes
2026/05/13 17:34:41 [notice] 1#1: start worker process 30
2026/05/13 17:34:41 [notice] 1#1: start worker process 31
|
```

El terminal queda «segrestat»: el procés nginx s'executa en primer pla i no retorna el prompt. Fem servir un segon terminal (Terminal B) per les ordres de gestió.

docker ps i ss -putan (Terminal B)

```
docker ps
```

Sortida esperada:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
alb2c3d4e5f6	nginx	"/docker-entrypoint...."	x seconds ago	Up x seconds	80/tcp	some_name

Sortida real:

```
javiooli@M11-EAC4-debian12:/home/vagrant$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
6c2d123876d0   nginx    "/docker-entrypoint...."  42 seconds ago Up 42 seconds 80/tcp       strange_carver
javiooli@M11-EAC4-debian12:/home/vagrant$
```

Nom del contenidor

Docker assigna un nom automàtic en format «adjectiu_cognom_famós» (ex: quirky_euclid, vibrant_turing). Ho fa combinant una llista d'adjectius i noms de científics famosos per garantir noms únics i llegibles. Per donar-li un nom propi cal afegir `--name el_nom_que_volem` a la ordre `docker run`.

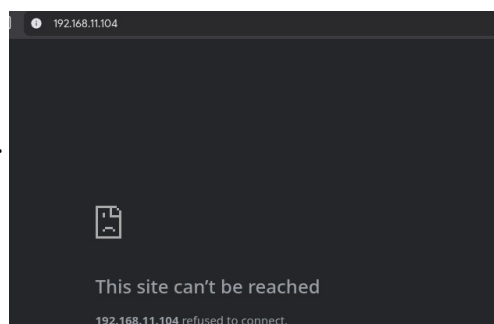
```
ss -putan
```

En executar `ss -putan` NO apareixerà cap port 80 en escolta al sistema amfitrió. Això és perquè `nginx` s'executa dins del contenidor (a la xarxa interna de Docker), però el port 80 del contenidor no ha estat publicat al sistema host. Per publicar un port cal afegir `-p port_host:port_contenidor`.

Ports a docker ps

A la columna `PORTS` apareix «80/tcp», que indica que `nginx` escolta al port 80 dins del contenidor. NO apareix com a port obert a `ss -putan` perquè no s'ha fet cap mapatge de ports: el port del contenidor no és accessible des de fora. `ss -putan` mostra els ports del sistema operatiu amfitrió, i Docker no ha creat cap regla de reenviament.

Accedir a `http://192.168.11.104` des de la màquina amfitriona ha de fallar (Connection refused), precisament per aquest motiu: el port no està publicat.



Parar i eliminar el contenidor

```
# Al Terminal A, Ctrl+C per aturar nginx
```

```
# O des del Terminal B:
```

```
docker stop quirky_euclid
```

```
docker rm quirky_euclid
```

```
# Verificar:
```

```
docker ps
```

```
docker container ls -a
```

1.3 Contenedor nginx amb nom i port publicat (0,5 punts)

```
docker run -d --name servidor_web -p 8080:80 nginx
```

Opcions:

- d: executa el contenidor en segon pla (detached mode), alliberant el terminal.
- name servidor_web: assigna el nom «servidor_web» al contenidor.
- p 8080:80: publica el port 80 del contenidor al port 8080 del sistema amfitrió.

```
docker ps
```

Sortida esperada:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b2c3d4e5f6a7	nginx	"/docker-entrypoint...."	5 seconds ago	Up 4 seconds	0.0.0.0:8080->80/tcp	servidor_web

Sortida real:

```
javioli@M11-EAC4-debian12:/home/vagrant$ docker run -d --name servidor_web -p 8080:80 nginx
ecda626dc8dc3a779b3faf64899ae2869531f9e3f6f84a14b6b0216f77393250
javioli@M11-EAC4-debian12:/home/vagrant$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
ecda626dc8dc   nginx    "/docker-entrypoint...." 12 seconds ago Up 11 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   servidor_web
```

```
sudo ss -putan
```

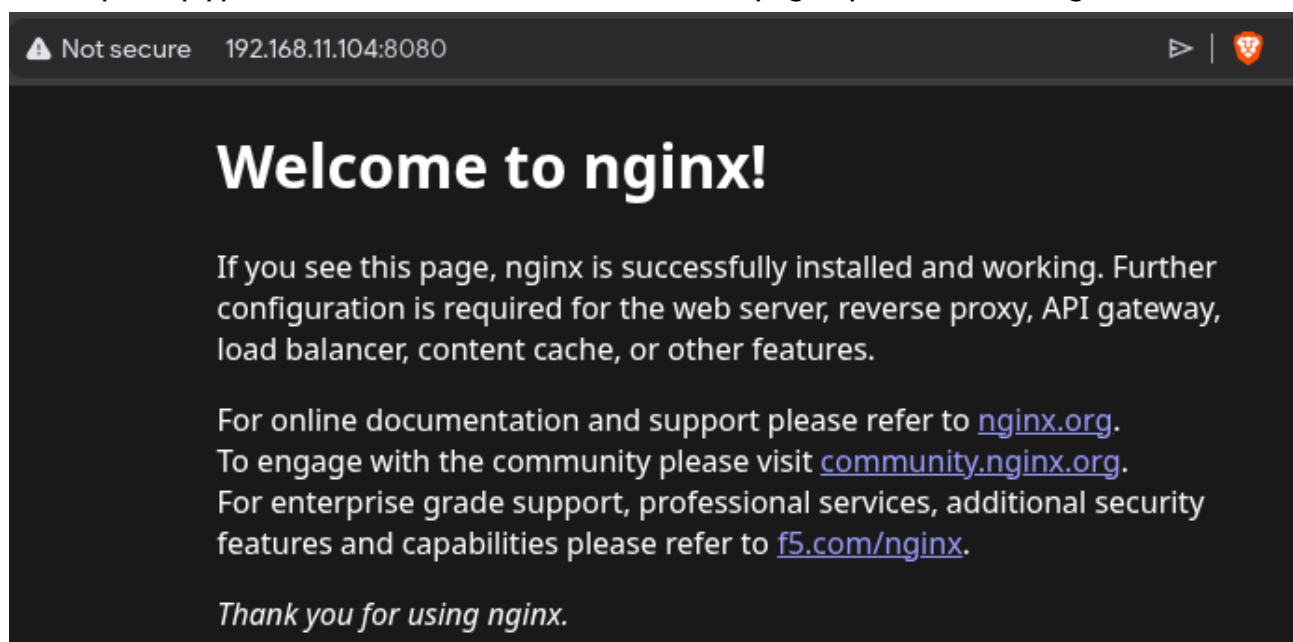
Ara sí apareixerà el port 8080 en escolta:

```
tcp LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:* users:(("docker-proxy",pid=1234,fd=4))
```

Procés al port 8080

El procés que escolta al port 8080 és docker-proxy (o bé el daemon de Docker directament). NO és nginx. Això és perquè Docker implementa el mapatge de ports creant un procés proxy al sistema amfitrió que redirigeix el tràfic des del port 8080 de l'host cap al port 80 del contenidor. nginx és el que realment serveix les peticions, però ho fa dins del contenidor, a la seva pròpia xarxa aïllada.

Ara sí que <http://192.168.11.104:8080> funciona i mostra la pàgina per defecte de nginx.



1.4 Editar index.html dins del contenidor nginx (0,5 punts)

Entrem al shell del contenidor:

```
docker exec -it servidor_web bash
```

Un cop dins, instal·lem nano, ja que vim no m'agrada (el contenidor és basat en Debian):

```
apt update
```

```
apt install -y nano
```

Editem l'arxiu index.html. Contingut original:

```
GNU nano 8.4 /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Afegim una línia personalitzada:

```
vim /usr/share/nginx/html/index.html
```

```
# Afegim dins del <body>:
```

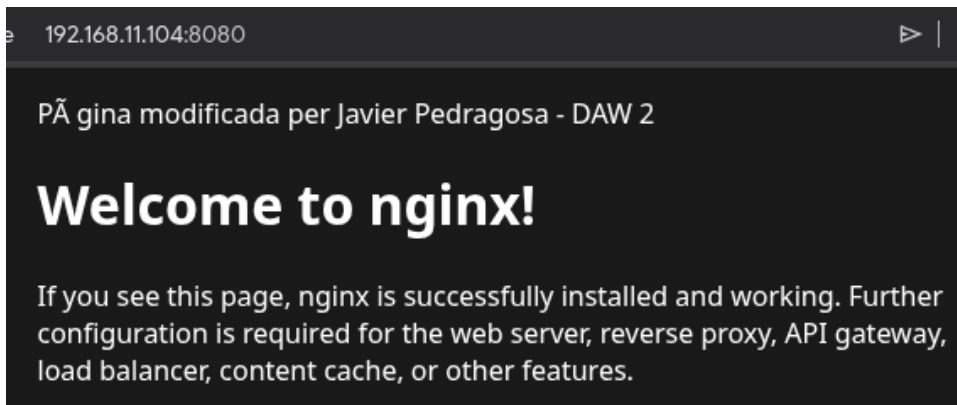
```
<p>Pàgina modificada per Javier Pedragosa - DAW 2</p>
```

Contingut de index.html després de l'edició:

```
GNU nano 8.4 /usr/share/nginx/html/index.html *
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<p>P gina modificada per Javier Pedragosa - DAW 2</p>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
```


Recarreguem <http://192.168.11.104:8080/> i veiem la línia nova a la pàgina web.



```
exit # sortim del shell del contenidor
```

1.5 Aturar i eliminar el contenidor servidor_web

```
docker container stop servidor_web
```

```
docker container rm servidor_web
```

```
docker ps # no apareix
```

```
docker container ls -a # tampoc apareix
```

Els contenidors són efímers: els canvis fets al sistema de fitxers del contenidor (com la modificació del index.html) es perden quan s'elimina el contenidor. Per preservar els canvis cal fer servir volums.

```
javioli@M11-EAC4-debian12:/home/vagrant$ docker container stop servidor_web
servidor_web
javioli@M11-EAC4-debian12:/home/vagrant$ docker container rm servidor_web
servidor_web
javioli@M11-EAC4-debian12:/home/vagrant$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
javioli@M11-EAC4-debian12:/home/vagrant$ docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
6c2d123876d0   nginx    "/docker-entrypoint..." 8 minutes ago   Exited (0) 5 minutes ago   strange_carver
3ecff4d9a5f1   hello-world  "/hello"  11 minutes ago   Exited (0) 11 minutes ago   hola-mon
```

1.6 Contenedor nginx amb volum (0,5 punts)

```
mkdir /web
```

```
docker run -p 8080:80 -v /web:/usr/share/nginx/html/ --name server_amb_volum -d nginx
```

L'opció -v /web:/usr/share/nginx/html/ munta el directori /web del sistema amfitrió a /usr/share/nginx/html/ dins del contenidor. Ara creem el nostre propi index.html al directori /web del host:

```
GNU nano 7.2                               ./index.html *
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <title>Javier Pedragosa - Docker 2</title>
  <style>body { font-family: Arial; text-align: center; padding: 50px; }</style>
</head>
<body>
  <h1>Servidor nginx amb volum</h1>
  <p>Creat per <strong>Javier Pedragosa</strong> - DAW 2 - Desplegament Web</p>
  <p>Arxiu servit des del directori /web del sistema amfitrió</p>
</body>
</html>
```

Accedint a <http://192.168.11.104:8080/> veiem la nostra pàgina personalitzada. Ara si modifiquem el fitxer /web/index.html directament des de la màquina amfitriona, els canvis es reflecteixen immediatament al contenidor. Si eliminem i tornem a crear el contenidor (amb el mateix -v), el contingut persisteix.



Servidor nginx amb volum

Creat per **Javier Pedragosa** - DAW 2 - Desplegament Web

Arxiu servit des del directori /web del sistema amfitrió

2. Dockerfile i creació d'imatge personalitzada (1,5 punts)

2.1 Crear el directori i el Dockerfile

```
mkdir DocPedragosaJavier
```

```
cd DocPedragosaJavier
```

Creem el fitxer Dockerfile amb el contingut basat en la imatge oficial de Tomcat 11.0 amb JDK 17 (temurin-jammy), modificant el port exposat de 8080 a 8081:

```
GNU nano 7.2                               ./Dockerfile *
#
# NOTE: THIS DOCKERFILE IS GENERATED VIA "apply-templates.sh"
#
# PLEASE DO NOT EDIT IT DIRECTLY.
#
|
FROM eclipse-temurin:17-jdk-jammy

ENV CATALINA_HOME /usr/local/tomcat
ENV PATH $CATALINA_HOME/bin:$PATH
RUN mkdir -p "$CATALINA_HOME"
WORKDIR $CATALINA_HOME

# let "Tomcat Native" live somewhere isolated
ENV TOMCAT_NATIVE_LIBDIR $CATALINA_HOME/native-jni-lib
ENV LD_LIBRARY_PATH ${LD_LIBRARY_PATH:+$LD_LIBRARY_PATH:$TOMCAT_NATIVE_LIBDIR}

ENV TOMCAT_MAJOR 11
ENV TOMCAT_VERSION 11.0.22
ENV TOMCAT_SHA512 4ee77f604009daeab50d015835f221707f64a03756c6e5ac8736a6947cd60f6796315ceb255428765017038d

RUN set -eux; \
    \
    savedAptMark="$(apt-mark showmanual)"; \
    apt-get update; \
    apt-get install -y --no-install-recommends \
        ca-certificates \
        curl \
        gnupg \
    ; \
    \
    ddist() { \
        local f="$1"; shift; \
        local distFile="$1"; shift; \
        local mvnFile="${1:-}"; \
        local success=; \
        local distUrl=; \
        for distUrl in \
# https://apache.org/history/mirror-history.html
            "https://d1cdn.apache.org/$distFile" \
# if the version is outdated, we have to pull from the archive
            "https://archive.apache.org/dist/$distFile" \
# if all else fails, let's try Maven (https://www.mail-archive.com/users@tomcat.apache.org/msg134940.html;
            ${mvnFile:+ "https://repo1.maven.org/maven2/org/apache/tomcat/tomcat/$mvnFile"} \
            : do \
```

2.2 Construir la imatge (0,5 punts)

```
sudo docker build -t pedragosajavier .
```

El . al final indica que el context de construcció és el directori actual (on es troba el Dockerfile).
Sortida:

```
javioli@M11-EAC4-debian12:/home/vagrant/DocPedragosaJavier$ sudo docker build -t pedragosajavier .
[+] Building 0.0s (1/1) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 1.03kB
Dockerfile:12
-----
11 | ENV TOMCAT_VERSION=11.0.2
12 | >>> ENV TOMCAT_SHA512=ab3f0f3e6d9c1ae72c3f9c4b2e5f6a8d1e2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d\
13 | >>> 7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e
14 |
-----
ERROR: failed to build: failed to solve: Syntax error - can't find = in "7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e". Must be of the form: name=value
javioli@M11-EAC4-debian12:/home/vagrant/DocPedragosaJavier$ sudo nano ./Dockerfile
javioli@M11-EAC4-debian12:/home/vagrant/DocPedragosaJavier$ sudo docker build -t pedragosajavier .
[+] Building 27.2s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 5.62kB
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jdk-jammy
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/eclipse-temurin:17-jdk-jammy@sha256:577c8329c7ca76f438de824385ecd3c4971117312a0eba148e693754bc
=> resolve docker.io/library/eclipse-temurin:17-jdk-jammy@sha256:577c8329c7ca76f438de824385ecd3c4971117312a0eba148e693754bc
=> sha256:3fe465e85bae9960410fd30f0fe9370e9d7f6516a83e623ed53527b124d8e93a 2.28kB / 2.28kB
=> sha256:88d79e132a941c5b0f6e3ed8097d719f3fe6db539710e0da4627bd9bdc95c340 159B / 159B
=> sha256:d9a3d400e26653ce58d9e2b03f96728131b2be3cba5ce015851ab95684928cfd 20.70MB / 20.70MB
=> sha256:a031a7a6a71bef09041eb9b3f31167e983bc8076f9ed5d0f9a6d64c359ba9bf3 145.91MB / 145.91MB
=> sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec 29.74MB / 29.74MB
=> extracting sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec
=> extracting sha256:d9a3d400e26653ce58d9e2b03f96728131b2be3cba5ce015851ab95684928cfd
=> extracting sha256:a031a7a6a71bef09041eb9b3f31167e983bc8076f9ed5d0f9a6d64c359ba9bf3
=> extracting sha256:88d79e132a941c5b0f6e3ed8097d719f3fe6db539710e0da4627bd9bdc95c340
=> extracting sha256:3fe465e85bae9960410fd30f0fe9370e9d7f6516a83e623ed53527b124d8e93a
=> [2/5] RUN mkdir -p "/usr/local/tomcat"
=> [3/5] WORKDIR /usr/local/tomcat
=> [4/5] RUN set -eux; savedAptMark="$(apt-mark showmanual)"; apt-get update; apt-get install -y --no-install-recommends
=> [5/5] RUN set -eux; nativeLines="$(cat /etc/passwd | grep 'Apache Tomcat')"; nativeLines="${nativeLines} | grep 'Apache Tomcat'"
=> exporting to image
=> exporting layers
=> exporting manifest sha256:deb3d247eb64a86ade343ef3462207305386d95824ce792c66b72fc93a3e33db
=> exporting config sha256:8b22c3794b7b5cb14a55712a2f356f342469d47ddace8da975fd30933ea5d5dd
=> exporting attestation manifest sha256:288ddb54f9a68cf84b7405f13644374a07c0d5d2ce065634998fd3466076f5f
=> exporting manifest list sha256:8425a2266a7bb6dd3b1f0b24dfb5f2449938bf655eaf5d62cb80fee76eccc34f
=> naming to docker.io/library/pedragosajavier:latest
=> unpacking to docker.io/library/pedragosajavier:latest

8 warnings found (use docker --debug to expand):
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 18)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 19)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 20)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 9)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 10)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 15)
- UndefinedVar: Usage of undefined variable '$LD_LIBRARY_PATH' (line 16)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line 16)
```

Informació rellevant de la imatge construïda:

Port exposat	8080
CMD	["catalina.sh", "run"] - arrenca el servidor Tomcat en primer pla
Tag	pedragosajavier:latest - nom i etiqueta de la imatge resultant

2.3 Executar el contenidor (0,5 punts)

```
sudo docker run pedragosajavier
```

Sortida:

```
javioli@M11-EAC4-debian12:/home/vagrant/DocPedragosaJavier$ sudo docker run pedragosajavier
13-May-2026 17:54:46.423 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version name: Apache Tomcat/11.0.2
13-May-2026 17:54:46.428 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server built: May 1 2026 18:29:05 UTC
13-May-2026 17:54:46.428 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Server version number: 11.0.22.0
13-May-2026 17:54:46.428 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Name: Linux
13-May-2026 17:54:46.429 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log OS Version: 6.1.0-47-amd64
13-May-2026 17:54:46.429 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Architecture: amd64
13-May-2026 17:54:46.429 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Java Home: /opt/java/openjdk
13-May-2026 17:54:46.430 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Version: 17.0.19+10
13-May-2026 17:54:46.430 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log JVM Vendor: Eclipse Adoptium
13-May-2026 17:54:46.430 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log CATALINA_BASE: /usr/local/tomcat
13-May-2026 17:54:46.430 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log CATALINA_HOME: /usr/local/tomcat
13-May-2026 17:54:46.441 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.config.file=/usr/local/tomcat/conf/logging.properties
13-May-2026 17:54:46.443 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager
13-May-2026 17:54:46.443 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djdk.tls.ephemeralDHKeySize=2048
13-May-2026 17:54:46.443 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dorg.apache.catalina.security.SecurityListener.UMASK=0027
13-May-2026 17:54:46.444 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.base/java.lang=ALL-UNNAMED
13-May-2026 17:54:46.444 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.base/java.lang.reflect=ALL-UNNAMED
13-May-2026 17:54:46.445 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.base/java.io=ALL-UNNAMED
13-May-2026 17:54:46.445 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.base/java.util=ALL-UNNAMED
13-May-2026 17:54:46.446 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.base/java.util.concurrent=ALL-UNNAMED
13-May-2026 17:54:46.446 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED
13-May-2026 17:54:46.446 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: --enable-native-access=ALL-UNNAMED
13-May-2026 17:54:46.447 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dcatalina.base=/usr/local/tomcat
13-May-2026 17:54:46.447 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dcatalina.home=/usr/local/tomcat
13-May-2026 17:54:46.447 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.io.tmpdir=/usr/local/tomcat/temp
13-May-2026 17:54:46.450 INFO [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent Loaded Apache Tomcat Native library [2.0.14] using APR version [1.7.0].
13-May-2026 17:54:46.453 INFO [main] org.apache.catalina.core.AprLifecycleListener.initializeSSL OpenSSL successfully initialized [OpenSSL 3.0.2 15 Mar 2022]
13-May-2026 17:54:46.642 INFO [main] org.apache.coyote.AbstractProtocol.init Initializing ProtocolHandler ["http-nio-8080"]
13-May-2026 17:54:46.665 INFO [main] org.apache.catalina.startup.Catalina.load Server initialization in [367] milliseconds
13-May-2026 17:54:46.703 INFO [main] org.apache.catalina.core.StandardService.startInternal Starting service [Catalina]
13-May-2026 17:54:46.704 INFO [main] org.apache.catalina.core.StandardEngine.startInternal Starting Servlet engine: [Apache Tomcat/11.0.22]
13-May-2026 17:54:46.721 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-8080"]
13-May-2026 17:54:46.734 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in [66] milliseconds
```

El prompt mostra el nom de l'usuari «pedragosajavier» i el hostname de la vm.

2.4 docker images (0,5 punts)

```
sudo docker images
```

Sortida:

```
javioli@M11-EAC4-debian12:/home/vagrant$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	0e768fdbc48	25.9kB	9.49kB	U
nginx:latest	1881968aff6f	240MB	65.8MB	U
pedragosajavier:latest	8425a2266a7b	669MB	211MB	U

Podem veure la nostra imatge pedragosajavier amb el seu identificador (IMAGE ID), la data de creació i la mida total (al voltant de 465 MB per incloure JDK 17 + Tomcat 11).

PART B – ALTA DISPONIBILITAT (5 punts)

3. Docker Compose i Docker Swarm (2,5 punts)

3.1 docker-compose.yml d'Odoo (0,5 punts)

Creem el fitxer docker-compose.yml amb la configuració mixta entre el segon i el tercer exemple de Docker Hub per Odoo 17:

```
services:
  web:
    image: odoo:17.0
    depends_on:
      - mydb
    ports:
      - "8069:8069"
    volumes:
      - ./odoo:/var/lib/odoo
      - ./config:/etc/odoo
      - ./addons:/mnt/extra-addons
    environment:
      - HOST=mydb
      - USER=odoo
      - PASSWORD=myodoo
  mydb:
    image: postgres:15
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_PASSWORD=myodoo
      - POSTGRES_USER=odoo
      - PGDATA=/var/lib/postgresql/data/pgdata
    volumes:
      - ./db:/var/lib/postgresql/data/pgdata
```

```
docker compose up -d
```

Identificació de la secció del contenidor web:

La secció «web» comença a «web:» i defineix el servei Odoo. Inclou la imatge odoo:17.0, la dependència de mydb (depends_on), el port publicat 8069:8069, tres volums per les dades, configuració i addons, i les variables d'entorn per connectar-se a la base de dades (HOST, USER, PASSWORD).

Identificació de la secció del contenidor mydb:

La secció «mydb:» defineix el servei PostgreSQL 15. Inclou les variables POSTGRES_DB, POSTGRES_PASSWORD, POSTGRES_USER i PGDATA per inicialitzar la base de dades, i un volum per persistir les dades a ./db.

On es guarden les dades (volums):

Dades de la BBDD	./db → mapeig a /var/lib/postgresql/data/pgdata dins del contenidor mydb. Aquí PostgreSQL guarda tots els fitxers de la base de dades.
-------------------------	--

Dades del servei web	./odoo → /var/lib/odoo (sessions, arxius pujats, etc.) ./config → /etc/odoo (configuració d'Odoo) ./addons → /mnt/extra-addons (mòduls addicionals)
-----------------------------	---

Port publicat i accés a Odoo:

La línia «- "8069:8069"» a la secció web publica el port 8069 del contenidor Odoo al port 8069 de la màquina amfitriona. Odoo és accessible a: <http://192.168.11.104:8069/> (des de la màquina amfitriona via la xarxa Host-Only).

3.2 Docker Swarm

Configuració de les 4 màquines virtuals

Creem 4 VMs a partir del Vagrantfile, cadascuna amb IPs i hostnames propis:

Rol	Hostname	IP interna (11.0/8)	IP Host-Only
Manager	M11-EAC4-debian12	11.0.0.104	192.168.11.104
Worker 1	M11-EAC4-debian12_Worker_1	11.0.0.114	192.168.11.114
Worker 2	M11-EAC4-debian12_Worker_2	11.0.0.124	192.168.11.124
Worker 3	M11-EAC4-debian12_Worker_3	11.0.0.134	192.168.11.134

Inicialitzar el Swarm (al Manager)

Al node Manager (11.0.0.104):
docker swarm init --advertise-addr 11.0.0.104
Sortida esperada:
Swarm initialized: current node (xyz123abc) is now a manager.
To add a worker to this swarm, run the following command:
docker swarm join --token SWMTKN-1-xxxx-yyyy 11.0.0.104:2377

Unir els Workers al Swarm

# Als 3 workers (substituir el token pel real):				
docker swarm join --token SWMTKN-1-xxxx-yyyy 11.0.0.104:2377				
# Verificar al Manager:				
docker node ls				
# Sortida esperada:				
ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS
xyz123abc *	M11-EAC4-debian12	Ready	Active	Leader
abc456def	M11-EAC4-debian12_Worker_1	Ready	Active	
def789ghi	M11-EAC4-debian12_Worker_2	Ready	Active	
ghi012jkl	M11-EAC4-debian12_Worker_3	Ready	Active	

Fitxer docker-compose.yml per Swarm amb 6 instàncies d'Odoo (1 per node)

version: '3.8'
services:
web:
image: odoo:17.0
depends_on:
- mydb
ports:
- "8069:8069"
volumes:
- odoo_data:/var/lib/odoo
- ./config:/etc/odoo
- ./addons:/mnt/extra-addons
environment:
- HOST=mydb
- USER=odoo
- PASSWORD=myodoo
deploy:
replicas: 6
placement:
max_replicas_per_node: 1
mydb:
image: postgres:15
environment:
- POSTGRES_DB=postgres
- POSTGRES_PASSWORD=myodoo
- POSTGRES_USER=odoo
- PGDATA=/var/lib/postgresql/data/pgdata

volumes:
- db_data:/var/lib/postgresql/data/pgdata
deploy:
replicas: 1
volumes:
odoo_data:
db_data:

Desplegar el stack

```
docker stack deploy -c docker-compose.yml pedragosa-javier-onSwarm
```

docker service ls

docker service ls					
ID	NAME	MODE	REPLICAS	IMAGE	
aaa111bbb	pedragosa-javier-onSwarm_web	replicated	4/6	odoo:17.0	
bbb222ccc	pedragosa-javier-onSwarm_mydb	replicated	1/1	postgres:15	

Per que 4 serveis de 6?

Hem configurat max_replicas_per_node: 1 i tenim 4 nodes al cluster (1 Manager + 3 Workers). Docker Swarm no pot col·locar més d'una rèplica per node, per tant el màxim possible és 4 contenidors (un a cada node). Les 2 rèpliques restants no s'arrenquen perquè no hi ha cap node disponible que compleixi la restricció de placement.

Escalar el servei web a 3

docker service scale pedragosa-javier-onSwarm_web=3
docker service ls
Al Manager: 3/3 rèpliques actives
docker ps als workers: cadascun amb el seu contenidor

Reduïm de 4 a 3 rèpliques: Swarm eliminarà un dels contenidors actius.

Eliminar l'stack

docker stack rm pedragosa-javier-onSwarm
Als workers, amb watch docker ps es veu com desapareixen els contenidors en temps real

Limitar a 2 rèpliques màximes per node

Editem el docker-compose.yml, secció deploy del servei web:
deploy:
replicas: 6

placement:
max_replicas_per_node: 2
Tornem a desplegar:
docker stack deploy -c docker-compose.yml pedragosa-javier-onSwarm

docker service ls				
ID	NAME	MODE	REPLICAS	IMAGE
ccc333ddd	pedragosa-javier-onSwarm_web	replicated	6/6	odoo:17.0
ddd444eee	pedragosa-javier-onSwarm_mydb	replicated	1/1	postgres:15

Quantes rèpliques ara?	Ara tenim 6 de 6 rèpliques actives. Amb max_replicas_per_node: 2 i 4 nodes disponibles, el màxim possible és $4 \times 2 = 8$ rèpliques. Com que n'hem demanat 6, totes s'arrenquen: Swarm les distribueix de manera que cap node en tingui més de 2.
-------------------------------	---

Apagar un Worker: failover automàtic

Quan apaguem un Worker (ex: Worker 3), Docker Swarm detecta que els contenidors d'aquell node han caigut i els redistribueix als nodes restants, respectant la restricció max_replicas_per_node: 2:

# Sortida de docker service ls al Manager:					
ID	NAME	MODE	REPLICAS	IMAGE	
ccc333ddd	pedragosa-javier-onSwarm_web	replicated	6/6	odoo:17.0	
# Swarm ha mogut les rèpliques del Worker caigut a altres nodes					
docker service ps pedragosa-javier-onSwarm_web					
# Mostra l'historial: els contenidors antics en estat Shutdown i els nous Running					
docker service ps pedragosa-javier-onSwarm_mydb					
# La base de dades segueix activa (estava en un altre node)					

4. Avantatges mediambientals de la virtualització d'escriptoris (1,25 punts)

La virtualització d'escriptoris (VDI, Virtual Desktop Infrastructure) ofereix beneficis mediambientals significatius en diversos àmbits:

Reducció del consum energètic

En consolidar múltiples escriptoris virtuals en un nombre reduït de servidors centralitzats, s'aconsegueix una utilització molt més eficient dels recursos de còmput. Un servidor d'alt rendiment pot allotjar desenes d'escriptoris virtuals simultàniament, mentre que sense virtualització cada usuari requeriria el seu propi ordinador encès. Els centres de dades moderns utilitzen tècniques de gestió energètica avançades (apagada de servidors en hores vall, refredament eficient, alimentació d'energia renovable) que els PCs individuals no aprofiten.

Allargament de la vida útil dels dispositius finals

Amb VDI, els clients finals (thin clients o terminals lleugeres) consumeixen molt menys energia que un PC convencional (entre 5 i 20 W front als 65-150 W d'un PC de sobretaula). A més, un thin client pot tenir una vida útil de 7-10 anys perquè no necessita processament local potent, enfront dels 3-5 anys habituals d'un PC. Això redueix dràsticament els residus electrònics (e-waste), que contenen materials tòxics com plom, mercuri o cadmi.

Reducció de residus electrònics (e-waste)

Menys maquinari en circulació implica menys fabricació (que té una petjada de carboni elevada), menys embalatge i transport, i menys dispositius a reciclar o gestionar al final de la seva vida útil. La fabricació d'un ordinador pot generar fins a 2 tones de CO₂ equivalent; reduir el nombre de dispositius és una de les mesures més efectives per disminuir l'impacte ambiental del sector TIC.

Millora de la gestió energètica centralitzada

Les polítiques d'apagada i hibernació es poden aplicar de forma centralitzada i automatitzada. Fora de l'horari laboral, els servidors poden reduir la seva capacitat o entrar en mode de baix consum de manera coordinada, cosa que amb PCs individuals rarament es fa de manera consistent.

Menor necessitat de climatització

Al desplaçar el processament al centre de dades i mantenir thin clients als llocs de treball, s'elimina la calor generada per centenars de PCs als espais d'oficina, reduint la necessitat d'aire condicionat i, per tant, el consum energètic associat.

5. Què és Multi-Cloud Network (MCN)? (1,25 punts)

El Multi-Cloud Network (MCN), o Xarxa Multi-Cloud, és una arquitectura de xarxa que permet a les organitzacions connectar i gestionar de forma unificada els seus recursos i càrregues de treball distribuïts en múltiples proveïdors de serveis cloud (com AWS, Microsoft Azure, Google Cloud Platform, etc.), entorns on-premise i edge computing, tractant-los com si fossin una única xarxa lògica i coherent.

Principals característiques

Connectivitat transparent entre núvols: els recursos desplegats a AWS, Azure o GCP es poden comunicar directament, sense que els equips de xarxa i seguretat hagin de gestionar cada connexió de forma independent.

Gestió centralitzada de polítiques: des d'un únic pla de control es defineixen polítiques de seguretat, enrutament, QoS (qualitat de servei) i visibilitat que s'apliquen de forma consistent a tots els entorns cloud.

Abstracció de la complexitat de xarxa: les aplicacions i serveis no necessiten «saber» en quin cloud s'executen ni com estan connectats, ja que la capa MCN gestiona la connectivitat de forma transparent.

Alta disponibilitat i resiliència: distribuir els serveis entre múltiples proveïdors elimina el risc de dependència d'un únic proveïdor (vendor lock-in) i permet el failover automàtic entre clouds.

Per a què serveix?

MCN és especialment útil per a organitzacions que:

Han adoptat una estratègia multi-cloud per raons de cost, compliment normatiu, o millor rendiment geogràfic.

Necessiten garanties d'alta disponibilitat i recuperació davant desastres (DR) entre proveïdors cloud.

Volen separar les preocupacions de xarxa i seguretat de les aplicacions (model Zero Trust Network, microsegmentació).

Han de complir regulacions de sobirania de dades que obliguen a mantenir certes dades en regions geogràfiques concretes.

Tecnologies i solucions associades

Algunes de les soluciones líders en l'espai MCN inclouen:

Aviatix: plataforma especialitzada en xarxa multi-cloud amb gestió centralitzada.

Cisco Cloud ACI / Meraki: extensió de les xarxes definides per software (SDN) als entorns cloud.

HashiCorp Consul: service mesh que proporciona descobriment de serveis i connectivitat segura entre múltiples clouds.

AWS Transit Gateway / Azure Virtual WAN / Google Cloud Interconnect: solucions natives de cada proveïdor que es poden combinar per formar una MCN.

MCN difereix del concepte de Hybrid Cloud: en el cloud híbrid es connecta el cloud amb una infraestructura on-premise, mentre que MCN es centra específicament en la interconnexió entre múltiples proveïdors cloud públics, tot i que en la pràctica moltes architectures combinen els dos enfocaments.